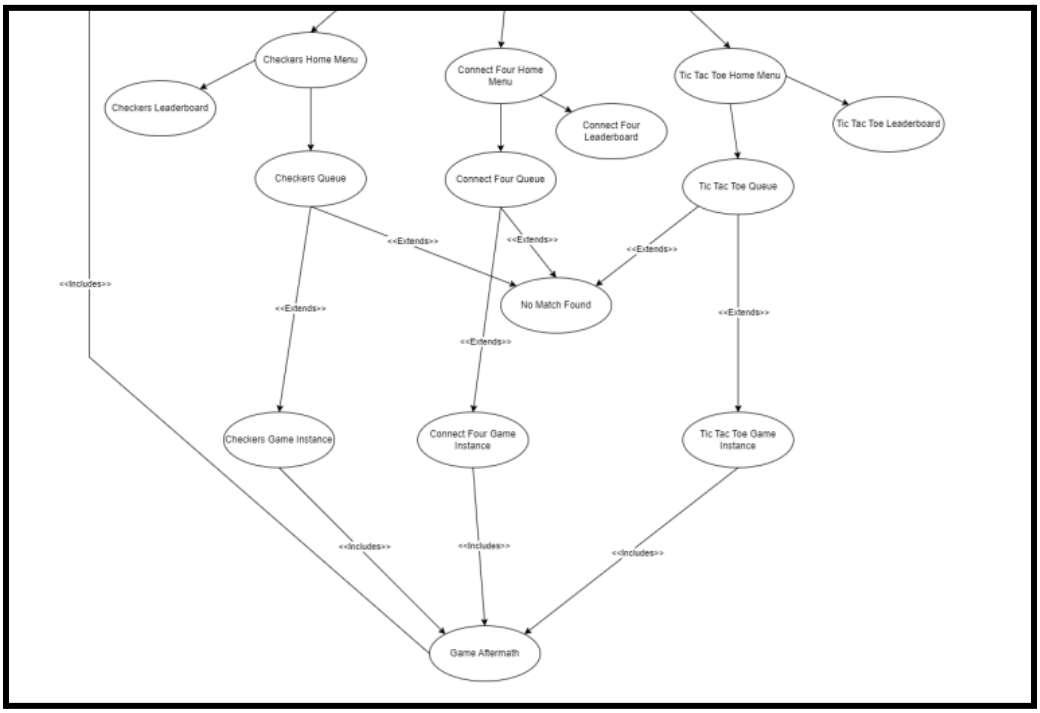
# CHANGES MADE

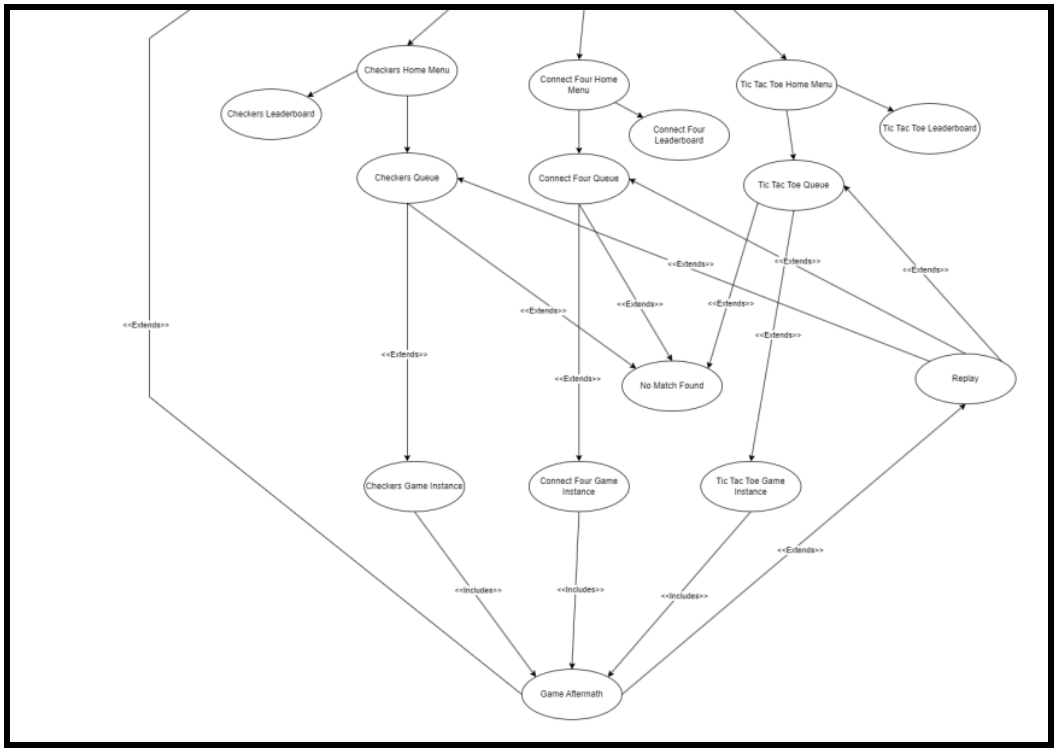
## Features Implemented:

We implemented a “replay” feature that allows players to revisit the game upon its completion. This feature was introduced in response to feedback provided by the Review 1 team, as it seemed like a logical feature for our game.

To show how we integrated this feature into our system, we updated the frontend use case diagram by incorporating a new use case "Replay Game." This use case extends the existing "Game Aftermath" use case. Additionally, we created a use case description to outline the functionality, preconditions, and expected outcomes associated with the replay feature.




*Before implementing “Replay”*




*After implementing "Replay"*

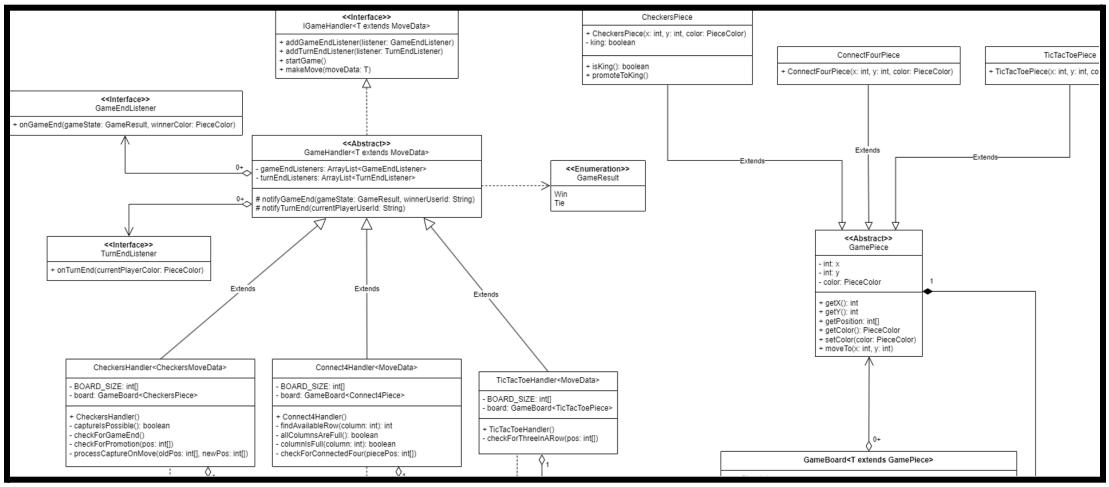
# Changes Made:

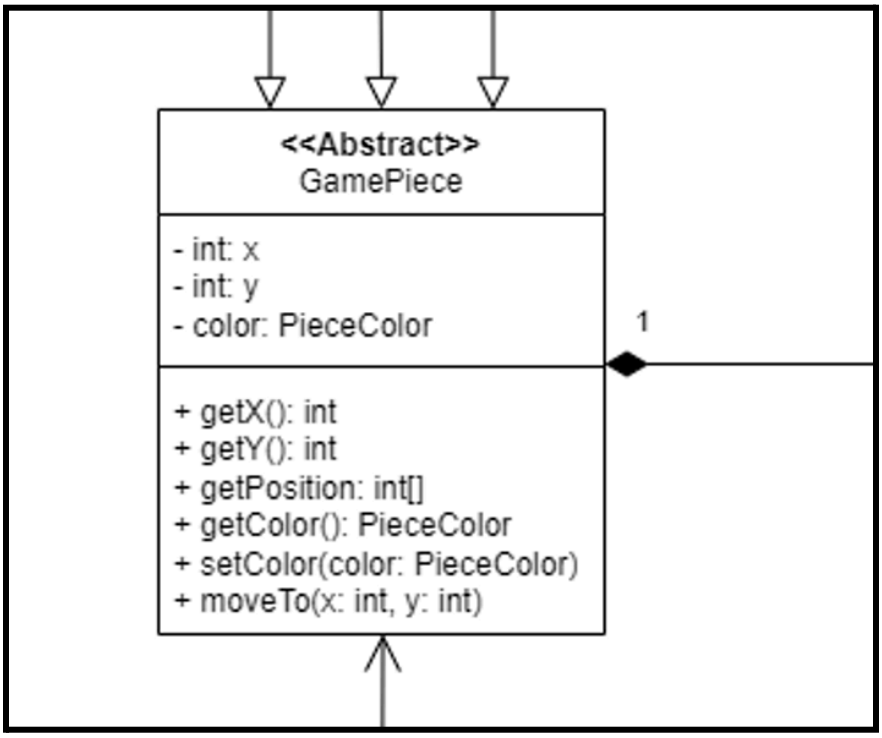
## Backend:

## Structure Diagram:

- ***Added dependency relationship between game handlers and their respective pieces (i.e. CheckersHandler -> CheckersPiece)***
- ***Fixed formatting error in <<Abstract>> GamePiece where the field x: int was instead formatted as int: x. This is now fixed***

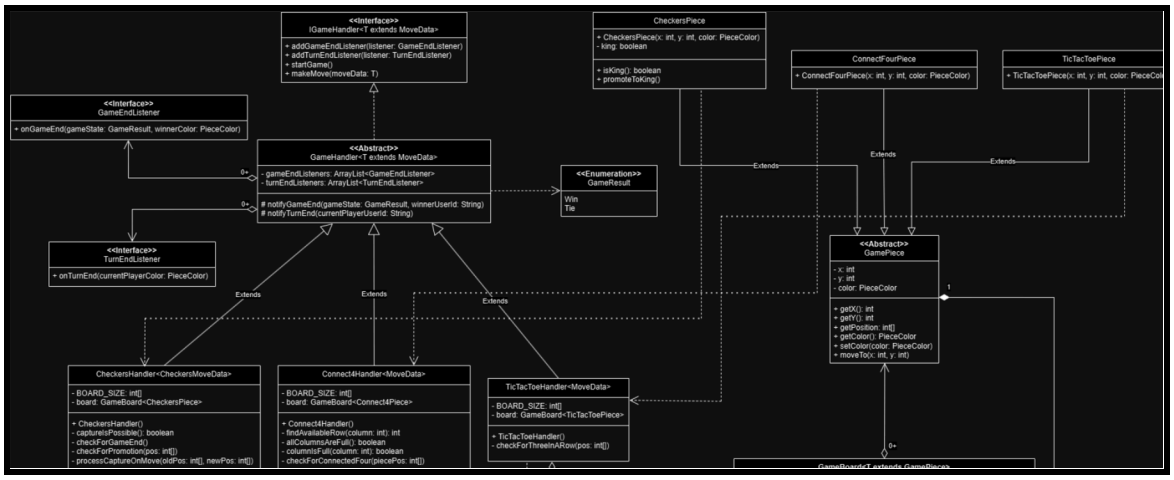
***Before:***

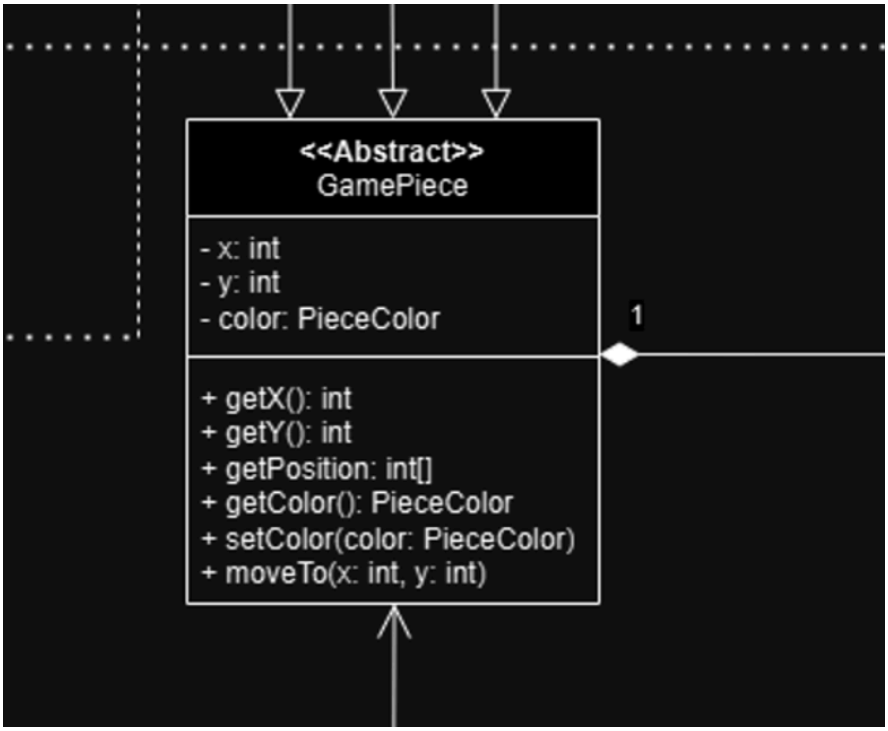



—

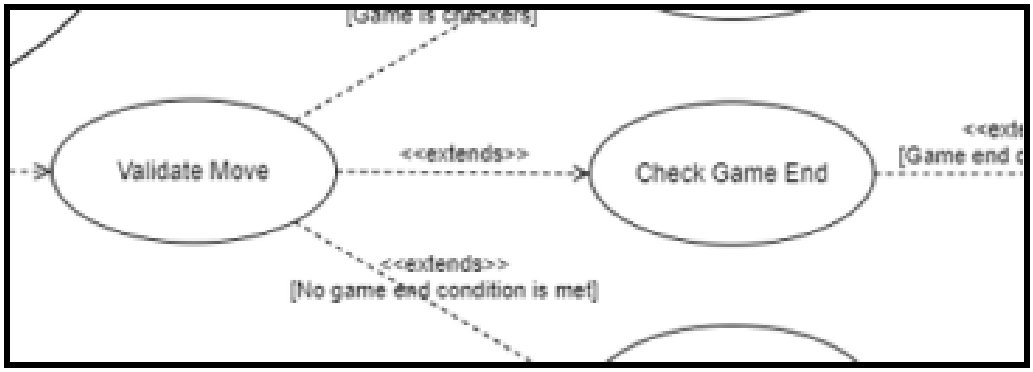
***After:***

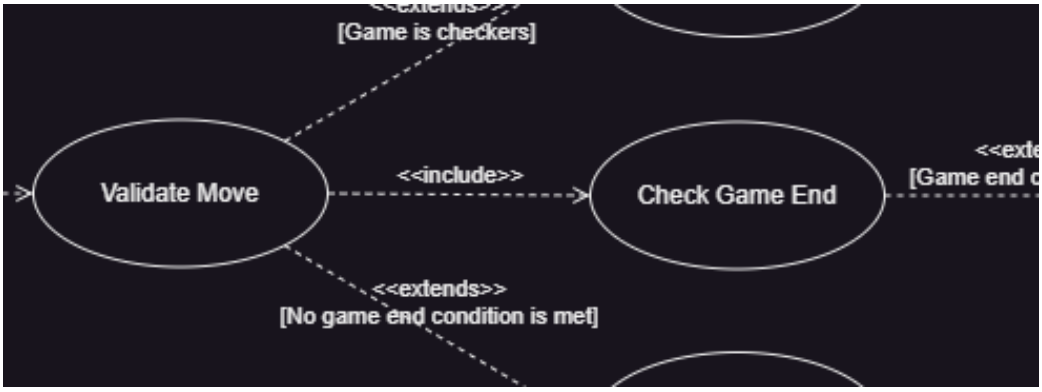



## Use-Case Diagram:

- ***Change relationship between validate move and check game end to <<include>> suggested by review 1***



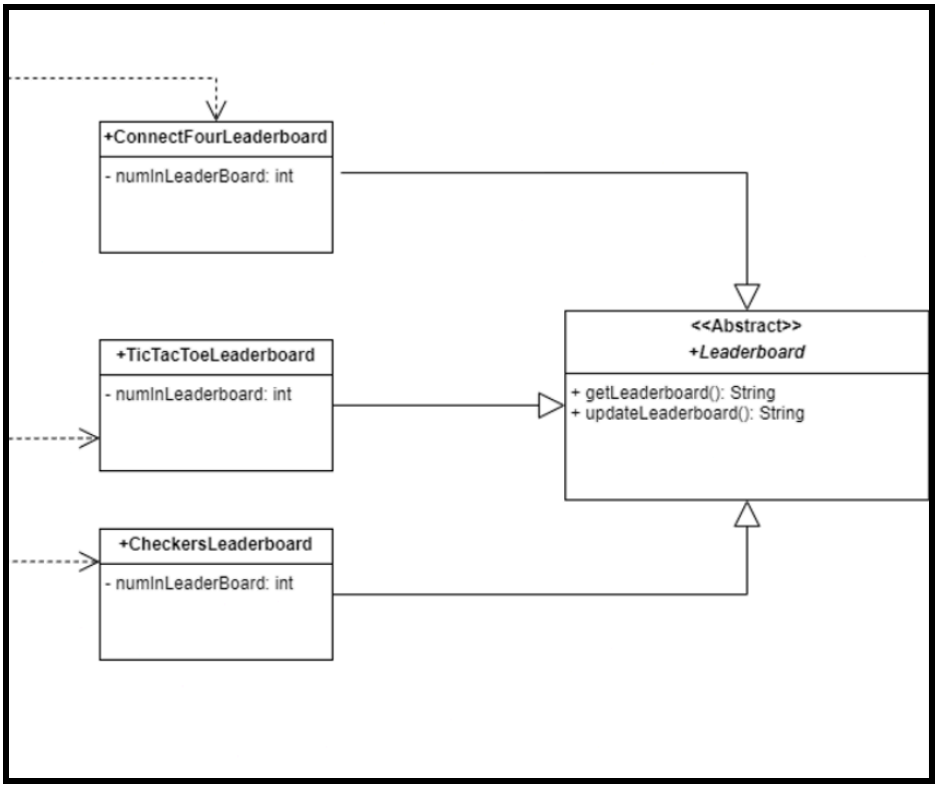



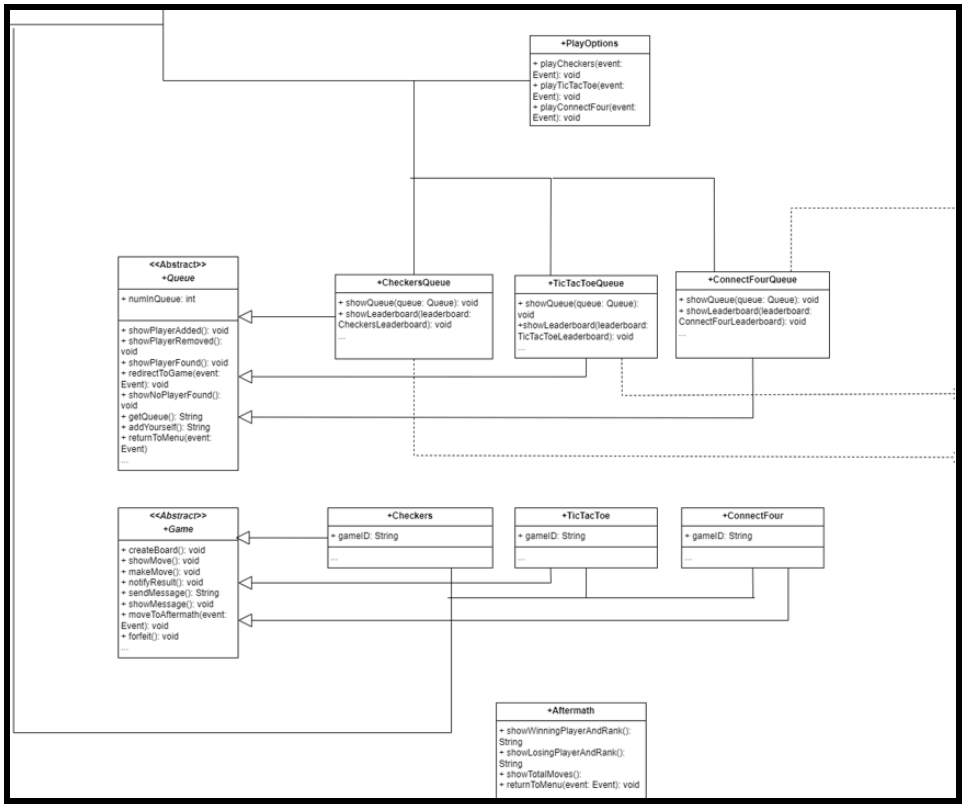

# Frontend

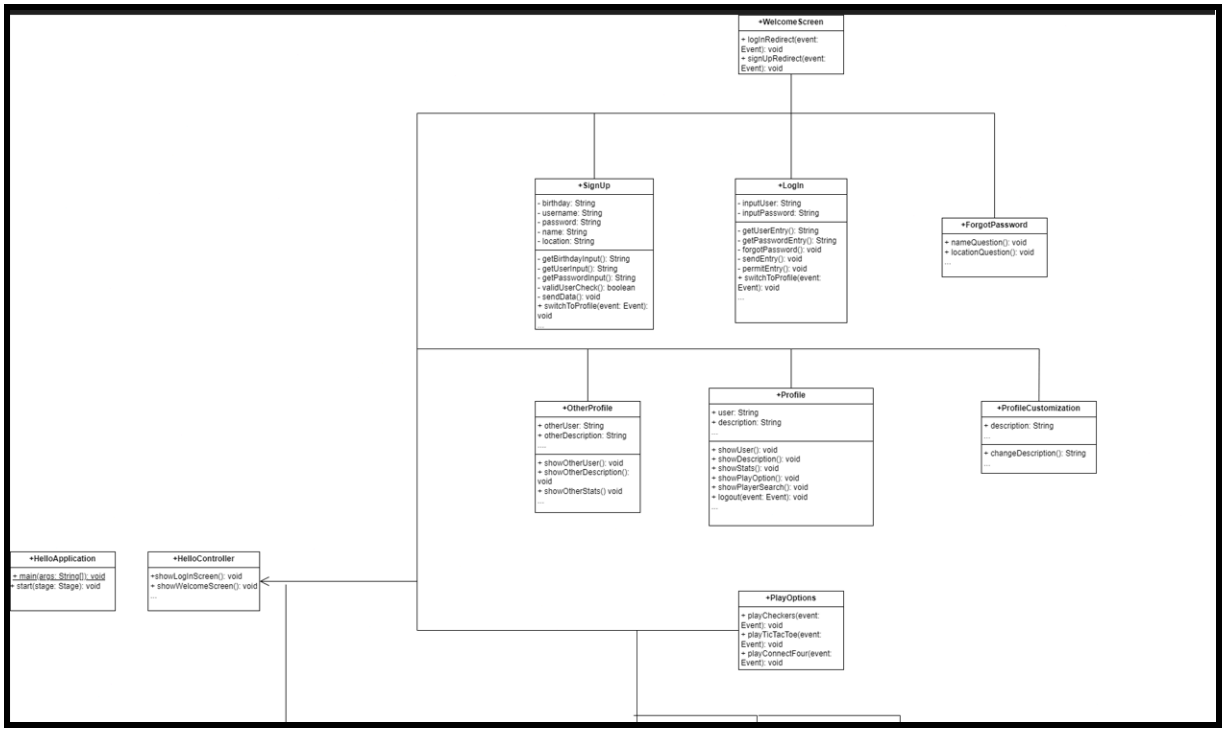
## Structure Diagram:

- ***Changed leaderboard implementation. Now there is one special controller for each game that is solely made for performing leaderboard calculations and displaying***
- ***Lots of controllers were added to the diagram. As examples, Menu controllers for each game and HowToPlay controllers for each game were added***
- ***Added new components, such as ClientReceiver to track specific networking actions***
- ***Reformulated lots of classes (doesn't need to go much in detail, just show Before3 and After3)***

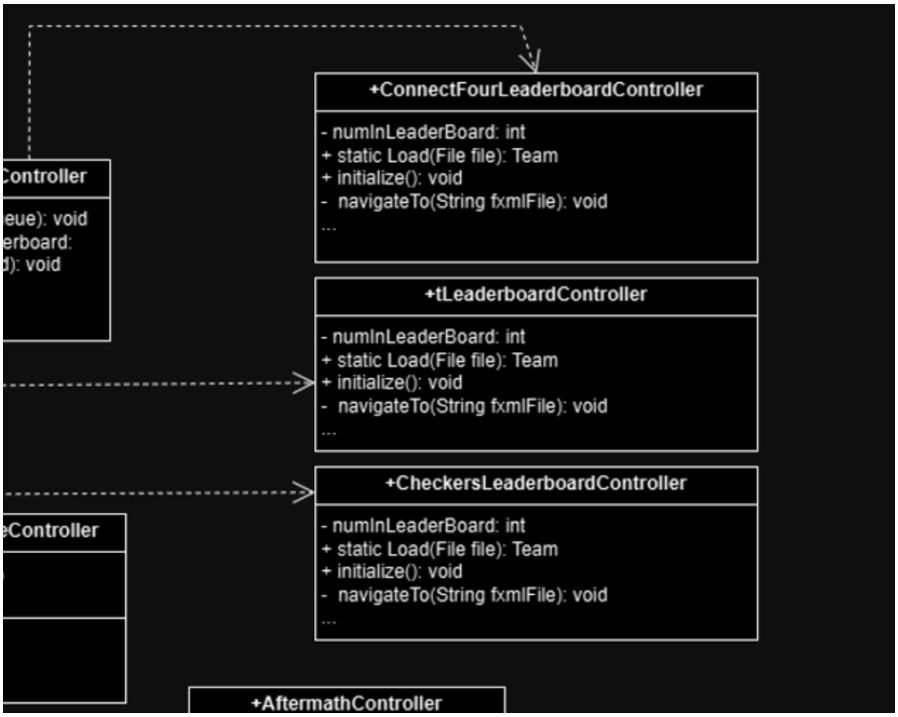
## Before:

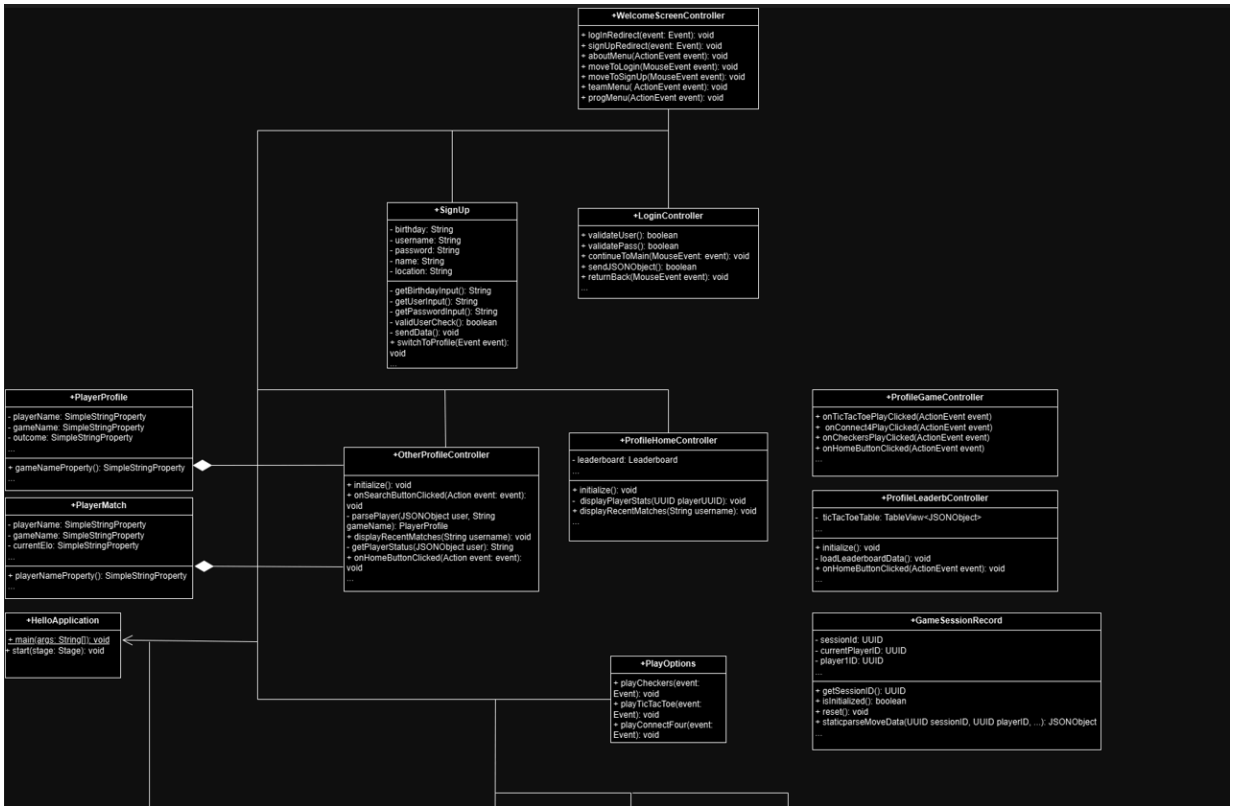

+ConnectFourLeaderboard
- numInLeaderBoard: int
+TicTacToeLeaderboard
- numInLeaderboard: int
+CheckersLeaderboard
- numInLeaderBoard: int
<<Abstract>>
+Leaderboard
+ getLeaderboard(): String
+ updateLeaderboard(): String

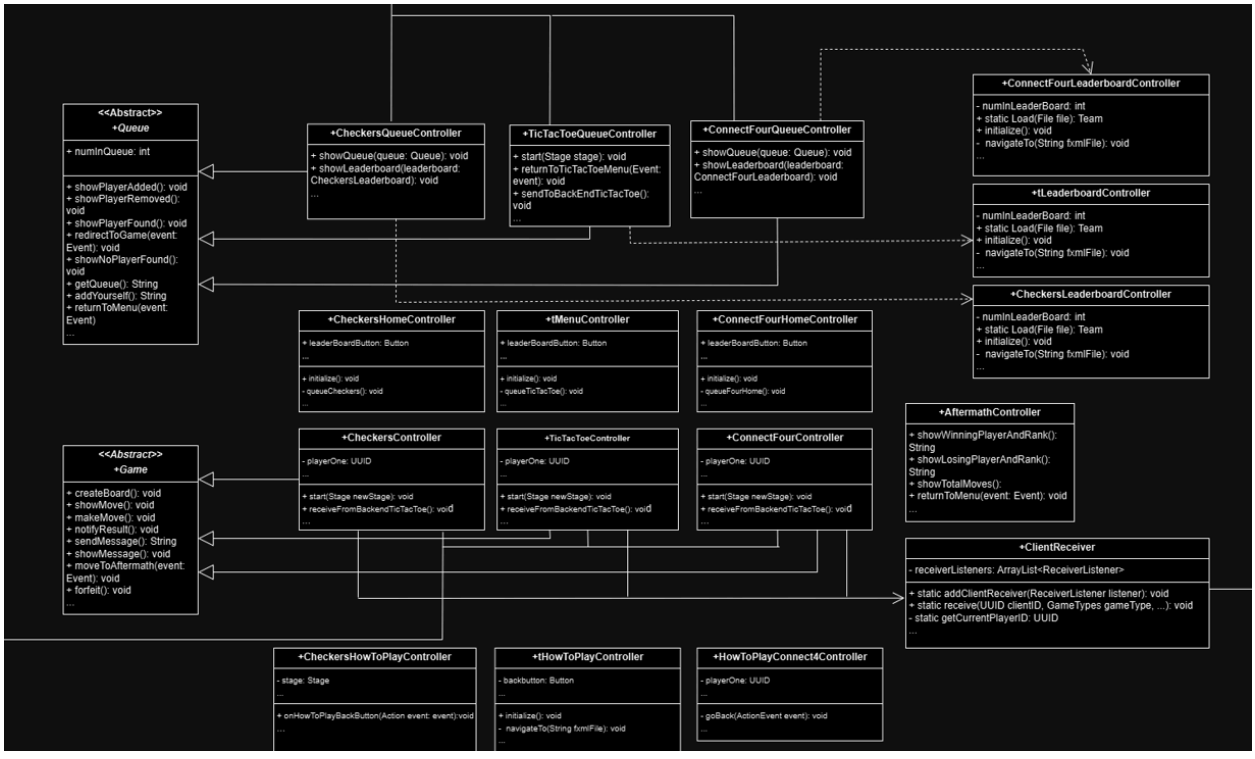

+PlayOptions
+ playCheckers(event: Event): void
+ playTicTacToe(event: Event): void
+ playConnectFour(event: Event): void
<<Abstract>>
+Queue
+ numInQueue: int
+ showPlayerAdded(): void
+ showPlayerRemoved(): void
+ showPlayerFound(): void
+ redirectToGame(event: Event): void
+ showNoPlayerFound(): void
+ getQueue(): String
+ addYourself(): String
+ returnToMenu(event: Event)
+CheckersQueue
+ showQueue(queue: Queue): void
+ showLeaderboard(leaderboard: CheckersLeaderboard): void
+TicTacToeQueue
+ showQueue(queue: Queue): void
+showLeaderboard(leaderboard: TicTacToeLeaderboard): void
+ConnectFourQueue
+ showQueue(queue: Queue): void
+ showLeaderboard(leaderboard: ConnectFourLeaderboard): void
<<Abstract>>
+Game
+ createBoard(): void
+ showMove(): void
+ makeMove(): void
+ notifyResult(): void
+ sendMessage(): String
+ showMessage(): void
+ moveToAftermath(event: Event): void
+ forfeit(): void
+Checkers
+ gameID: String
+TicTacToe
+ gameID: String
+ConnectFour
+ gameID: String
+Aftermath
+ showWinningPlayerAndRank(): String
+ showLosingPlayerAndRank(): String
+ showTotalMoves():
+ returnToMenu(event: Event): void


+WelcomeScreen
+ loginRedirect(event: Event): void
+ signUpRedirect(event: Event): void
+SignUp
- birthday: String
- username: String
- password: String
- name: String
- location: String
- getBirthdayInput(): String
- getUserInput(): String
- getPasswordInput(): String
- validUserCheck(): boolean
- sendData(): void
+ switchToProfile(event: Event): void
+LogIn
- inputUser: String
- inputPassword: String
- getUserEntry(): String
- getPasswordEntry(): String
- forgotPassword(): void
- sendEntry(): void
- permitEntry(): void
+ switchToProfile(event: Event): void
+ForgotPassword
+ nameQuestion(): void
+ locationQuestion(): void
+OtherProfile
+ otherUser: String
+ otherDescription: String
+ showOtherUser(): void
+ showOtherDescription(): void
+ showOtherStats() void
+Profile
+ user: String
+ description: String
+ showUser(): void
+ showDescription(): void
+ showStats(): void
+ showPlayOption(): void
+ showPlayerSearch(): void
+ logout(event: Event): void
+ProfileCustomization
+ description: String
+ changeDescription(): String
+HelloApplication
+ main(args: String[]): void
+ start(stage: Stage): void
+HelloController
+showLoginScreen(): void
+ showWelcomeScreen(): void
+PlayOptions
+ playCheckers(event: Event): void
+ playTicTacToe(event: Event): void
+ playConnectFour(event: Event): void

# After:


+ConnectFourLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
...
+tLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
...
+CheckersLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
...
+AftermathController


+WelcomeScreenController
+ loginRedirect(event: Event): void
+ signUpRedirect(event: Event): void
+ aboutMenu(ActionEvent event): void
+ moveToLogin(MouseEvent event): void
+ moveToSignUp(MouseEvent event): void
+ teamMenu( ActionEvent event): void
+ progMenu(ActionEvent event): void
+SignUp
- birthday: String
- username: String
- password: String
- name: String
- location: String
- getBirthdayInput(): String
- getUserInput(): String
- getPasswordInput(): String
- validUserCheck(): boolean
- sendData(): void
+ switchToProfile(Event event): void
+LoginController
+ validateUser(): boolean
+ validatePass(): boolean
+ continueToMain(MouseEvent event): void
+ sendJSONObject(): boolean
+ returnBack(MouseEvent event): void
+PlayerProfile
- playerName: SimpleStringProperty
- gameName: SimpleStringProperty
- outcome: SimpleStringProperty
+ gameNameProperty(): SimpleStringProperty
+OtherProfileController
+ initialize(): void
+ onSearchButtonClicked(Action event: event): void
- parsePlayer(JSONObject user, String gameName): PlayerProfile
+ displayRecentMatches(String username): void
- getPlayerStatus(JSONObject user): String
+ onHomeButtonClicked(Action event: event): void
+ProfileHomeController
- leaderboard: Leaderboard
+ initialize(): void
+ displayPlayerStats(UUID playerUUID): void
+ displayRecentMatches(String username): void
+ProfileGameController
+ onTicTacToePlayClicked(ActionEvent event)
+ onConnect4PlayClicked(ActionEvent event)
+ onCheckersPlayClicked(ActionEvent event)
+ onHomeButtonClicked(ActionEvent event)
+ProfileLeaderbController
- ticTacToeTable: TableView<JSONObject>
+ initialize(): void
- loadLeaderboardData(): void
+ onHomeButtonClicked(ActionEvent event): void
+PlayerMatch
- playerName: SimpleStringProperty
- gameName: SimpleStringProperty
- currentElo: SimpleStringProperty
+ playerNameProperty(): SimpleStringProperty
+HelloApplication
+ main(args: String[]): void
+ start(stage: Stage): void
+PlayOptions
+ playCheckers(event: Event): void
+ playTicTacToe(event: Event): void
+ playConnectFour(event: Event): void
+GameSessionRecord
- sessionId: UUID
- currentPlayerID: UUID
- player1ID: UUID
+ getSessionID(): UUID
+ isInitialized(): boolean
+ reset(): void
+ staticparseMoveData(UUID sessionID, UUID playerID, ...): JSONObject


<<Abstract>>
+Queue
+ numInQueue: int
+ showPlayerAdded(): void
+ showPlayerRemoved(): void
+ showPlayerFound(): void
+ redirectToGame(event: Event): void
+ showNoPlayerFound(): void
+ getQueue(): String
+ addYourself(): String
+ returnToMenu(event: Event)
+CheckersQueueController
+ showQueue(queue: Queue): void
+ showLeaderboard(leaderboard: CheckersLeaderboard): void
+TicTacToeQueueController
+ start(Stage stage): void
+ returnToTicTacToeMenu(Event: event): void
+ sendToBackEndTicTacToe(): void
+ConnectFourQueueController
+ showQueue(queue: Queue): void
+ showLeaderboard(leaderboard: ConnectFourLeaderboard): void
+ConnectFourLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
+tLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
+CheckersLeaderboardController
- numInLeaderBoard: int
+ static Load(File file): Team
+ initialize(): void
- navigateTo(String fxmlFile): void
+CheckersHomeController
+ leaderBoardButton: Button
+ initialize(): void
- queueCheckers(): void
+tMenuController
+ leaderBoardButton: Button
+ initialize(): void
- queueTicTacToe(): void
+ConnectFourHomeController
+ leaderBoardButton: Button
+ initialize(): void
- queueFourHome(): void
+AftermathController
+ showWinningPlayerAndRank(): String
+ showLosingPlayerAndRank(): String
+ showTotalMoves():
+ returnToMenu(event: Event): void
<<Abstract>>
+Game
+ createBoard(): void
+ showMove(): void
+ makeMove(): void
+ notifyResult(): void
+ sendMessage(): String
+ showMessage(): void
+ moveToAftermath(event: Event): void
+ forfeit(): void
+CheckersController
- playerOne: UUID
+ start(Stage newStage): void
+ receiveFromBackendTicTacToe(): void
+TicTacToeController
- playerOne: UUID
+ start(Stage newStage): void
+ receiveFromBackendTicTacToe(): void
+ConnectFourController
- playerOne: UUID
+ start(Stage newStage): void
+ receiveFromBackendTicTacToe(): void
+ClientReceiver
- receiverListeners: ArrayList<ReceiverListener>
+ static addClientReceiver(ReceiverListener listener): void
+ static receive(UUID clientID, GameTypes gameType, ...): void
- static getCurrentPlayerID: UUID
+CheckersHowToPlayController
- stage: Stage
+ onHowToPlayBackButton(Action event: event):void
+tHowToPlayController
- backbutton: Button
+ initialize(): void
- navigateTo(String fxmlFile): void
+HowToPlayConnect4Controller
- playerOne: UUID
- goBack(ActionEvent event): void

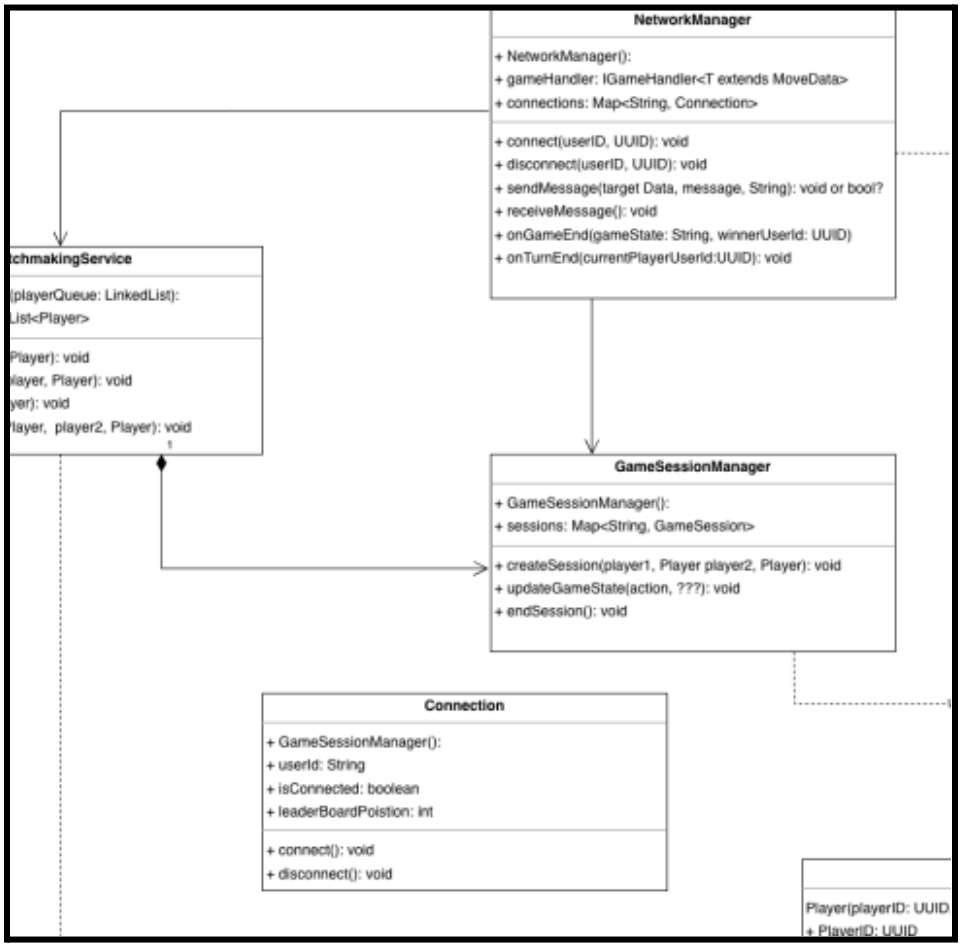
## Use-Case Diagram:

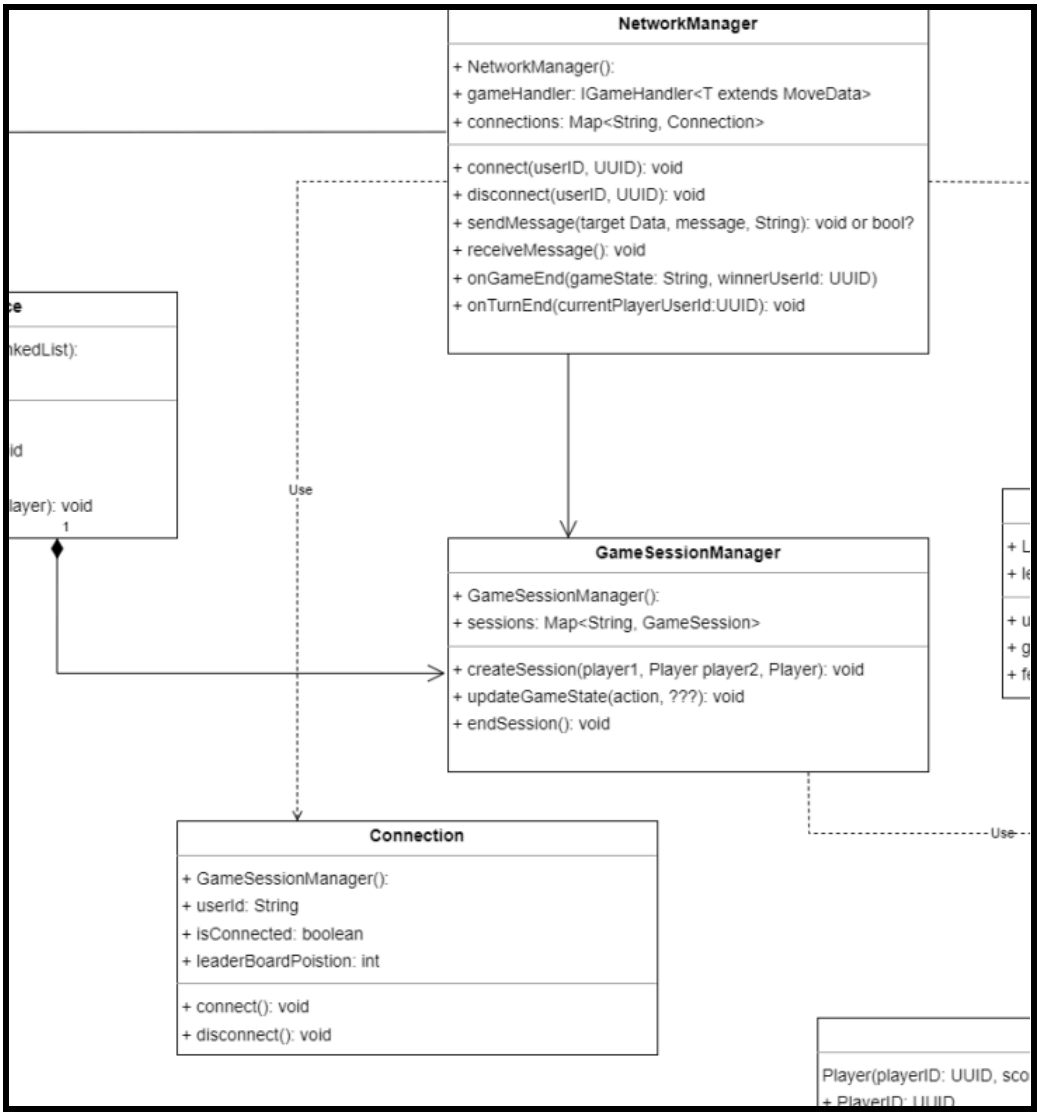
- ***Added replay use case to the diagram, and added associated use case descriptions suggested by review 1 - See "Features Implemented"***

# Networking

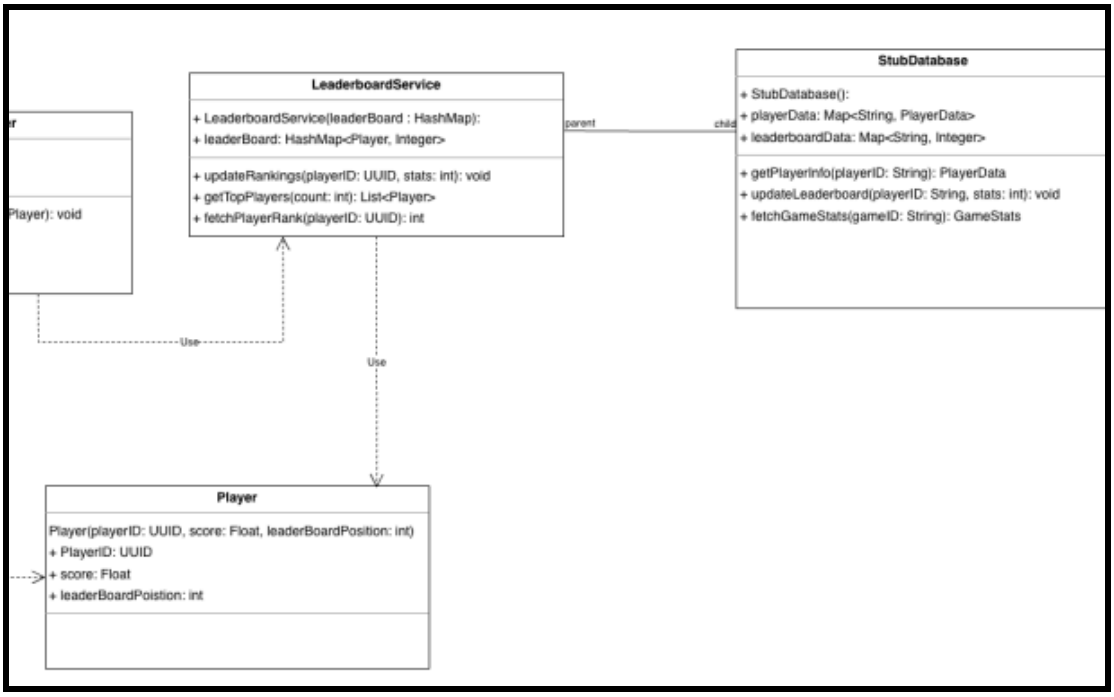
## Structure Diagram:

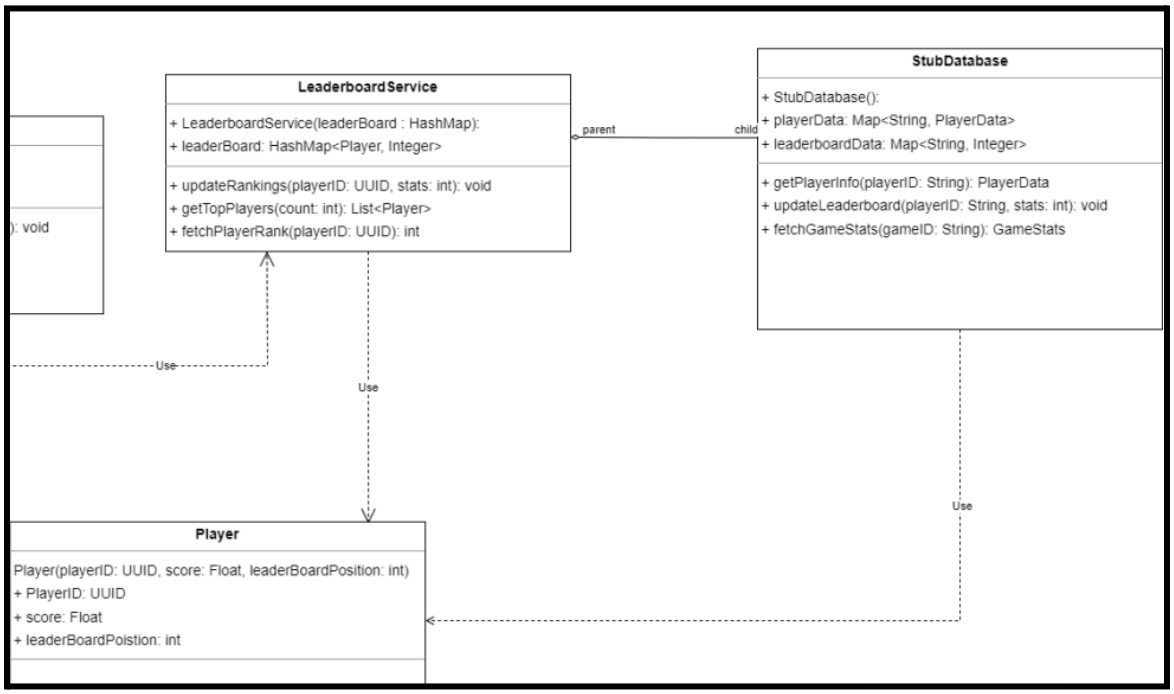
- ***Added relationship between networkManager and Connection suggested by review 2***






- ***Added relationship between StubDatabase and Player suggest by review 2***

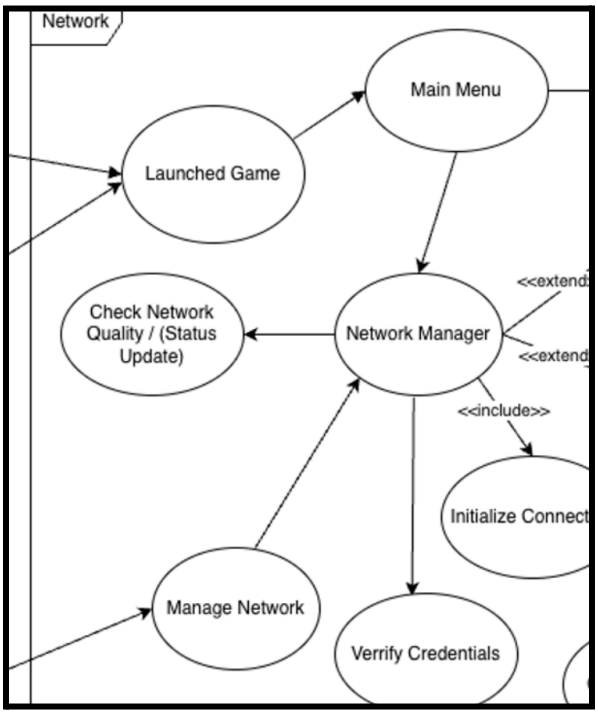


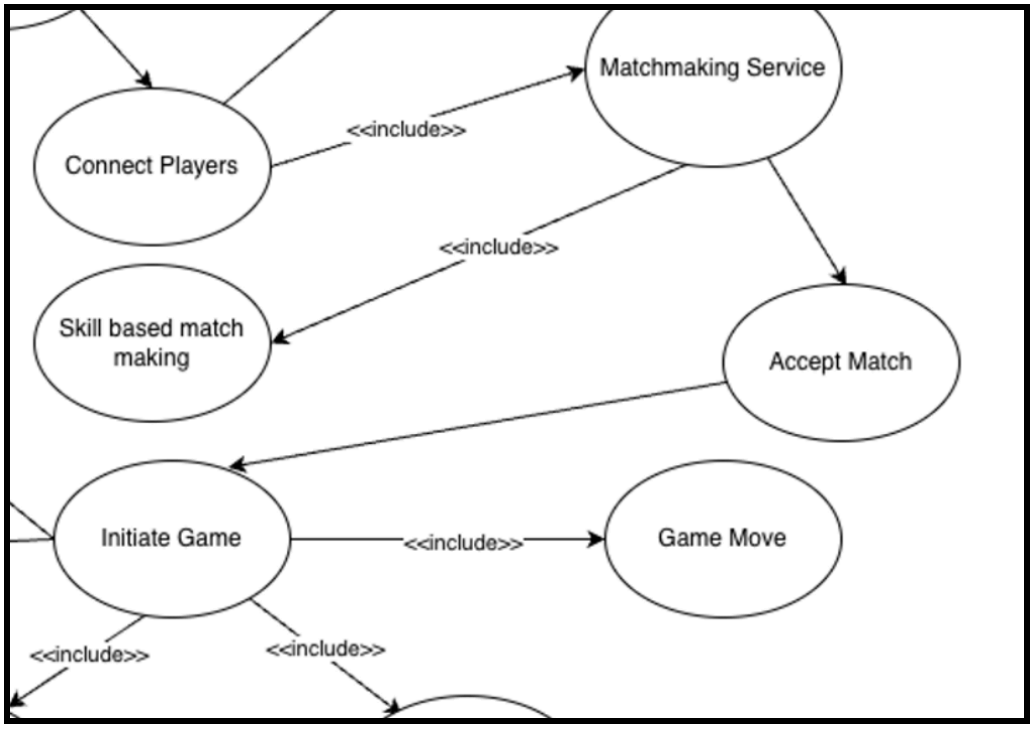




## Use-Case Diagrams:

- ***Minor change: added <<includes>> and <<excludes>> to certain connections (doesn't need to go much in detail, just show Before1, Before2, After1, and After2)***

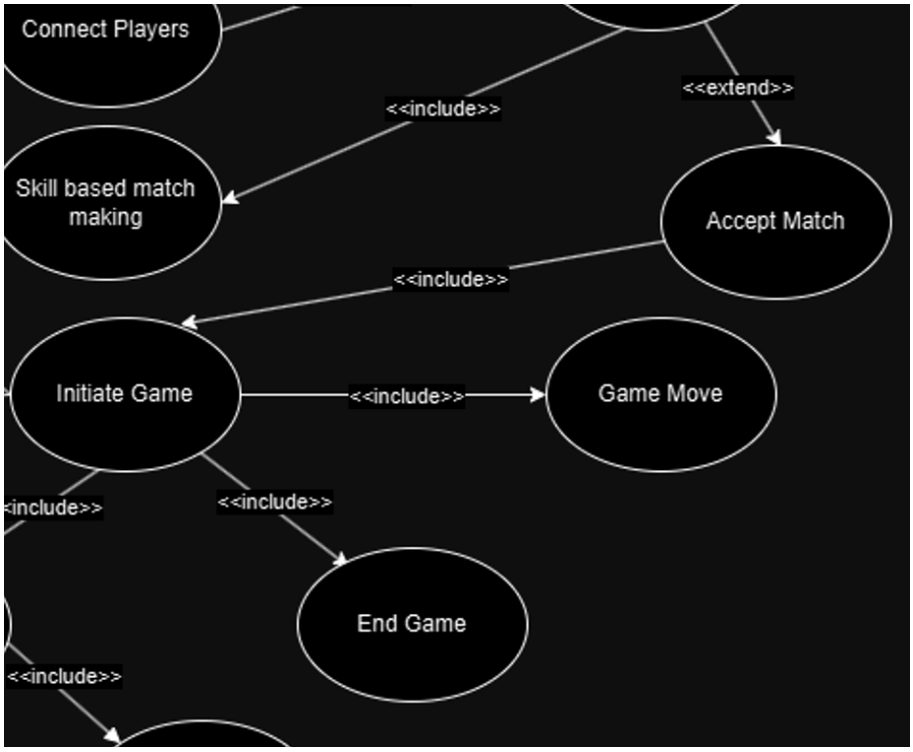
# Before:

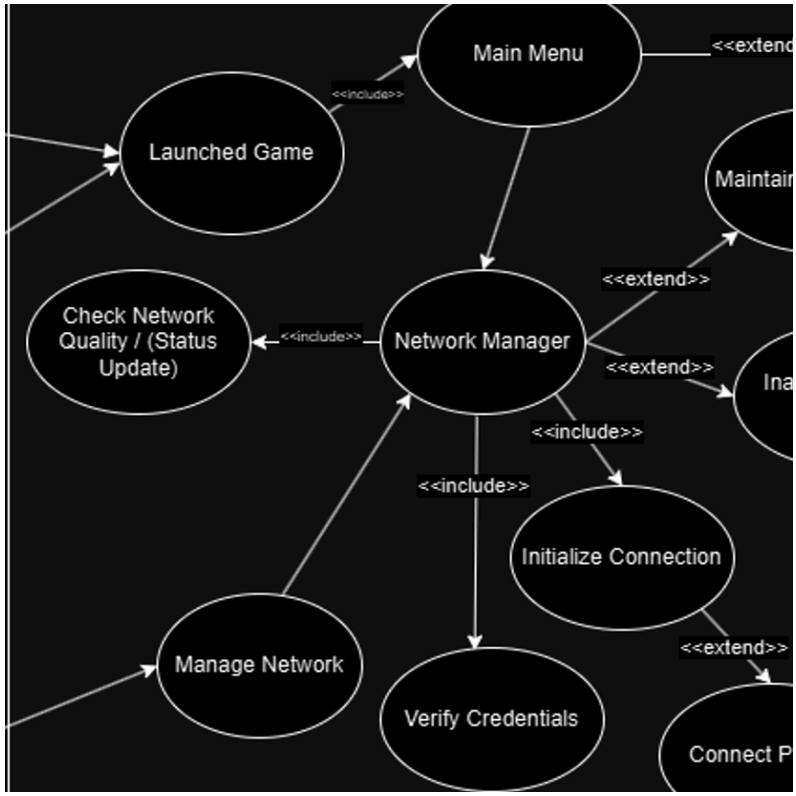

Network
Main Menu
Launched Game
Check Network Quality / (Status Update)
Network Manager
<<include>>
Initialize Connect
Manage Network
Verrify Credentials


Matchmaking Service
Connect Players
<<include>>
<<include>>
Skill based match making
Accept Match
Initiate Game
<<include>>
Game Move
<<include>>
<<include>>

–

**After:**


Connect Players
<<include>>
<<extend>>
Skill based match making
Accept Match
<<include>>
Initiate Game
<<include>>
Game Move
<<include>>
<<include>>
End Game
<<include>>


Main Menu
<<extend
<<include>>
Launched Game
Maintain
<<extend>>
Check Network Quality / (Status Update)
<<include>>
Network Manager
<<extend>>
Ina
<<include>>
<<include>>
Initialize Connection
<<extend>>
Manage Network
Verify Credentials
Connect P